產品功能說明

xConsole 模型主控台

統一記憶體 AI 機的模型編組與記憶體准入控制

文件名稱	xConsole 模型主控台　主要功能說明
版本	1.0
發行日期	2026/10/22
編製單位	云碩科技股份有限公司（xCloudinfo）
聯絡窗口	李偉業　jefflee@cloudinfo.com.tw
文件密級	機密　Confidential

云碩科技股份有限公司

2026/10/22

目錄

第一章　系統定位 3

1.1　這套系統解決的問題 3

1.2　適用機型 3

1.3　與既有監控工具的分工 4

第二章　主要功能 6

2.1　記憶體准入控制 6

2.2　角色欄位 7

2.3　情境切換 7

2.4　網頁換模型 8

2.5　即時監控 8

2.6　記錄檢視 9

第三章　三種記憶體架構 10

第四章　部署模式 11

4.1　單機模式 11

4.2　多機模式 11

4.3　目前的部署狀態 11

第五章　安全設計 12

第六章　實測數據 13

6.1　兩顆模型的對照 13

6.2　兩項與直覺相反的發現 13

6.3　上下文長度與顯存 14

6.4　畫面長度 14

附錄一　操作入口 15

附錄二　已知限制 16

第一章　系統定位

1.1　這套系統解決的問題

地端 AI 機器上同時跑著好幾個模型與服務：寫程式用的語言模型、抓網頁用的瀏覽器、語音辨識、語音合成、影片處理。過去要知道哪台機器開了什麼、還剩多少記憶體、能不能再多開一個，只能 SSH 進去逐台查，再靠記憶或記事本記下每個模型的參數。機器一多，這份紀錄很快就和現況脫節。

xConsole 把這件事收進一個網頁。它以「角色」而不是以「模型名稱」來排版——主腦、副腦、聽覺、說話、記憶、判斷、雙手、眼睛各占一格，空著的格子會直接顯示出來，一眼就看得出這台機器還缺什麼。

不過，這套系統真正要解決的不是「看」，而是「擋」。**統一記憶體機器一旦把記憶體吃爆，不是單一服務當掉，而是整台卡死、要走到機器前面按電源重開。** 我們的 HP Z2 Mini（Ryzen AI 395）就發生過不只一次。xConsole 在載入任何模型之前，會先試算「載下去之後最壞剩多少」，不夠就擋下來並用白話說明原因。

1.2　適用機型

xConsole 專為三個系列的統一記憶體機器設計。這三個系列的共同點是 CPU 與 GPU 共用同一塊記憶體，也正是傳統「只看顯存」的監控工具最容易判斷錯誤的地方。

表 1-1　適用機型與記憶體架構

系列	代表機型	記憶體配置	在 xConsole 中的架構代號
AMD Ryzen AI 395	HP Z2 Mini G1a	BIOS 切 96 GiB 給 GPU，系統只剩 31 GiB	unified_split
NVIDIA Jetson / 研華 MIC-743	Jetson AGX Thor、AGX Orin、Orin NX	統一記憶體單一池（Thor 為 122 GiB）	unified_single
NVIDIA DGX Spark	GB10	統一記憶體單一池（約 121 GiB）	unified_single

機群中的傳統多卡伺服器（例如 MI50 農場、P100、RTX 6000 Ada）也能納管，系統以 discrete 架構處理它們，但這不是本產品的主要對象。

一台機器也能管，十台也能管。賣一台 AI 機器給客戶時，可以把主控台直接裝在那台機器上，客戶開瀏覽器就能用；之後機器變多，再把它們串到同一個主控台，各機上的設定完全不必重做。

1.3　與既有監控工具的分工

云碩機群原本已有 amd-cluster-monitor 與 gpufleet 兩套監控儀表板，可以看到每台機器的 CPU、記憶體、磁碟與 GPU 使用率。xConsole 不重做這一層——它的硬體數據直接取自各機上已經在跑的監控代理，自己只負責監控做不到的部分：哪個角色由哪個模型擔任、載入前的記憶體試算、模型的啟停與更換、情境切換，以及各服務的記錄。

表 1-2　xConsole 與既有監控工具的分工

能力	既有監控儀表板	xConsole
硬體使用率、溫度、磁碟	有	沿用監控代理的數據
依角色排列的模型欄位	無	有
載入前的記憶體准入試算	無	有
網頁上啟動、停止、更換模型	無	有
一鍵切換整組參數（情境）	無	有
檢視各服務的記錄	無	有

第二章　主要功能

2.1　記憶體准入控制

每次啟動、重啟或更換模型之前，xConsole 都會先做一次試算。它把「目前還剩多少」、「這個模型需要多少」與「這台機器的安全底線」放在一起比較，只要任何一個記憶體池在載入後會跌破底線，就擋下並說明是哪一塊不夠、差多少。

試算看的是「載下去之後最壞剩多少」，而不是「現在剩多少」。兩者的差別在於，一個模型剛載入時用量往往不高，等到真正處理長請求時才把記憶體撐開；只看當下用量，就會放行一個稍後會把機器拖垮的載入。

對已經在跑的模型做重啟時，系統會把它自己目前占用的記憶體先算回可用量。這一點看似細節，卻是實際踩過的問題：若不這樣處理，重啟一個本來就跑得好好的模型會被它自己擋住。

以 2026/10/20 在 .26 上的實測為例，兩種情境的判斷如下。

表 2-1　准入控制實測判斷

情境	系統判斷	系統給出的理由
重啟已在運作的主腦模型	放行	會先釋放它自己占用的 63.9 GiB 顯存與 16.2 GiB 系統記憶體
在同一台再多載一顆 64 GiB 的模型	擋下	系統記憶體只剩 17 GiB，載下去會剩 3.2 GiB，低於 15 GiB 底線

被擋下時，畫面會說明原因，並提供「我知道風險，仍要執行」的選項。這是刻意保留的——系統的職責是讓人在知情的狀況下做決定，而不是完全剝奪操作者的判斷。

模型需要多少記憶體，系統的估算依序採用三種來源：這台機器實際載入過該模型時量到的數值、以目前運作中模型的實測值按權重大小比例推算、最後才是保守的預設估計。模型成功載入後，系統會自動量一次真實用量並記下來，下次就不必再猜。

2.2　角色欄位

每台機器的頁面以角色排列，而不是以模型名稱排列。角色分為主腦、副腦、嵌入記憶、語音辨識、語音合成、路由判斷、產圖雙手、抓料眼睛等。每個欄位卡片會顯示目前由哪個模型擔任、量化格式、能力標籤（文字、語音、影片、網頁）、服務埠號、上下文長度、顯存與系統記憶體用量，以及實測的速度區間。

速度數字全部取自引擎自己的記錄，不是手填的規格值。系統持續讀取引擎每次處理請求後留下的紀錄，累積成「匯入提示」與「生成回答」兩組速度區間，並記下量過的最長提示長度。

服務狀態不只分「開」與「關」。啟動中、停止中、失敗都會分別標示，因為卡在啟動中的服務若被歸為「關」會讓人以為它沒在跑，若被歸為「開」又會讓人以為它已經好了，兩種都會誤導判斷。

標記為「關鍵服務」的欄位，在停止或重啟時會多一道確認，並顯示這個服務停掉之後誰會受影響。例如 .26 的主腦欄位就註明了「同事 Samuel 的 Copilot 後端，停掉他就不能寫程式」。

2.3　情境切換

同一顆模型在同一台機器上，用途不同時需要的參數也不同。寫程式的用戶端一輪就會送出四萬個 token 以上的提示，需要很長的上下文；抓網頁清洗資料則每段只有幾千個 token，把並行數讓出來換吞吐量反而更划算。

表 2-2　預設情境

情境	上下文長度	並行數	適用用途
寫程式後端	131,072	1	GitHub Copilot 這類一次送大量程式碼脈絡的用戶端
抓料清洗	32,768	2	網頁抓取後的資料清洗，每段提示較短

切換情境時，系統會先備份目前的設定檔，再寫入新參數並重啟服務。若新設定載入失敗，系統自動還原成上一版並重新啟動，不會留下一個改壞又起不來的狀態。

2.4　網頁換模型

使用者可以直接在欄位卡片上按「換模型」，從這台機器已安裝的模型中挑選，不必 SSH 進機器、也不必知道設定檔放在哪裡。這個功能讓非管理者也能自行更換，例如寫程式的同仁可以在通用模型與程式專用模型之間自己切換。

換模型換的是一整份設定，而不只是兩個檔案路徑。每顆模型搭配的專家剖析檔、分片設定、快取策略都不一樣；若只替換權重路徑，引擎會因為參數對不上而在數秒內崩潰。因此系統只列出具備完整專屬設定檔的模型，套用時以該模型的設定為基礎，只保留與機器本身相關的項目（引擎執行檔、函式庫路徑、服務埠號、並行數）。

套用流程依序為：備份設定、套用新模型設定、執行記憶體准入試算、重新啟動、等待服務真正回應。**判斷是否成功，看的是服務埠能否正常回應模型清單，而不是啟動指令有沒有回傳成功。** 這是實測得來的教訓：啟動指令只要被系統接受就會回報成功，模型載入失敗卻是幾秒之後才發生，若只看指令結果，系統會誤以為切換成功而不去還原。

為了避免有人藉這個功能指向任意檔案，系統只接受掃描清單內的模型組合。2026/10/20 的實測中，送出指向 `/etc/passwd` 的請求會被直接拒絕，回應為「這個組合不在本機掃到的模型清單內」。

2.5　即時監控

每台機器的頁面同時顯示記憶體與硬體狀態。記憶體以堆疊長條呈現，每個運作中的模型各占一段顏色，並在長條上標出安全底線的位置，一眼就看得出距離危險還有多遠。統一記憶體單一池的機器只畫一條長條；切成兩塊的機器則分別畫出顯存與系統記憶體，因為對這類機器而言，系統記憶體那一塊才是真正會出事的地方。

硬體區塊顯示 GPU 使用率、溫度、功耗與時脈，CPU 負載與核心數，開機天數，各感測器溫度，以及磁碟用量。另有一張「監控中的埠」清單，列出這台機器上所有正在監聽的服務埠，並標示它屬於系統、模型或主控台，以及是否對外開放。

總覽頁把所有納管機器並列，顯示每台的記憶體用量、運作中的角色與目前情境；系統記憶體接近底線的機器會以醒目顏色標示。畫面每 15 秒自動更新一次。

2.6　記錄檢視

記錄頁可以選擇任一台機器的任一個服務，直接檢視它的系統記錄與引擎記錄。過去要查一個服務為什麼起不來，得先 SSH 進機器、找出記錄檔位置、再用指令過濾；現在在網頁上選好機器與服務即可看到。記錄區塊固定高度並在區塊內捲動，不會讓整個頁面無限延長。

第三章　三種記憶體架構

xConsole 最核心的設計，是把三種完全不同的記憶體配置分開計算。套用同一條公式，在某些機器上會盯錯數字，給出一個看起來安全、實際上危險的判斷。

表 3-1　三種記憶體架構的計算方式

架構代號	代表機型	計算方式	預設底線
unified_split	AMD Ryzen AI 395	顯存與系統記憶體分開計算，兩者都必須在底線之上	系統記憶體 15 GiB、顯存 2 GiB
unified_single	Jetson、DGX Spark	一個池子合併計算，模型需求與系統需求加總後比較	12 GiB
discrete	多卡伺服器	每張顯示卡各自計算，系統記憶體另外計算	系統記憶體 8 GiB、顯存 1 GiB

Ryzen AI 395 是最容易誤判的一種。BIOS 把 96 GiB 切給 GPU，作業系統只剩 31 GiB，因此顯存看起來很寬鬆，真正會耗盡的是那 31 GiB 的系統記憶體。過去造成整台當機的事件，都是系統記憶體耗盡而非顯存不足。15 GiB 的底線與機群既有的記憶體看門狗採用同一條標準。

Jetson 與 DGX Spark 則相反，它們的 `nvidia-smi` 在記憶體總量欄位回報的是 `[N/A]`，因為統一記憶體並不以傳統顯存的形式呈現。xConsole 對這類機器改讀作業系統的記憶體資訊，並把 GPU 與系統的需求合併計算，不會分開判斷成「兩邊各自都還夠」。

各機的底線可以在該機的欄位登錄檔中個別調整，以配合不同機型的實際狀況。

模型的上下文快取（KV）占用多少記憶體，也因模型而異，差距可達數個數量級。目前使用的 Qwen3.8-Flash-Next 採用線性混合注意力架構，每個 token 只需 1,056 位元組，128K 上下文全開也只有 0.13 GiB；傳統全注意力模型同樣長度則可能占用數十 GiB。xConsole 允許每個欄位指定自己模型的每 token 用量，不會一體套用同一個數字。

第四章　部署模式

4.1　單機模式

單機模式把節點代理與主控台裝在同一台機器上，這台機器只管理自己。這是賣一台 AI 機器給客戶時的標準做法：客戶拿到機器、開瀏覽器就能看到自己機器上的模型狀態並進行操作，不需要另外準備一台管理用的電腦。部署只需要一行指令。

4.2　多機模式

多機模式由一台機器擔任主控台，其餘機器各自執行節點代理。主控台彙整各機狀態，以群組方式呈現在同一個畫面上。由單機模式擴充為多機模式時，各機上的代理與欄位登錄完全不必重做，只需把新機器加入主控台的納管清單。

4.3　目前的部署狀態

表 4-1　目前納管的機器（2026/10/22）

機器	群組	記憶體架構	角色
.27（192.168.11.27）	AMD Strix Halo	unified_split	主控台所在機器，同時為抓料清洗引擎
.26（192.168.11.26）	AMD Strix Halo	unified_split	寫程式後端（同事 Copilot 使用）
.20（192.168.11.20）	Jetson / MIC-743	unified_single	醫美廣告影片產線

主控台網址為 `https://192.168.11.27:9400`。

第五章　安全設計

xConsole 能夠啟動與停止其他服務，因此在設計上把它當成一套需要防護的系統，而不是內部小工具。

表 5-1　安全設計

面向	做法
連線加密	主控台只提供 HTTPS。第一次啟動若沒有正式憑證，會自動產生暫用自簽憑證，不會退回只開 HTTP
登入	密碼以 PBKDF2-SHA256（200,000 輪）加鹽雜湊後存檔，不保存明文；密碼錯誤時固定延遲一秒，降低暴力猜測的效益
機器間的權杖	各機代理的權杖存放於該機的檔案中（權限 0600），只有主控台伺服器讀得到；瀏覽器只拿到簽章過的登入票證，看不到任何代理權杖
權杖傳遞	權杖只經由 Authorization 標頭傳遞，不放進網址，避免被存取記錄或瀏覽器歷史留下
網頁防護	回應帶有內容安全政策、禁止被嵌入框架、禁止 MIME 類型猜測等標頭
換模型的路徑限制	只接受本機掃描清單內的模型組合，拒絕指向任意路徑的請求
資源上限	代理與主控台都設有記憶體與 CPU 用量上限，避免它們本身成為壓垮機器的原因
破壞性操作	停止關鍵服務、強制略過准入試算、停止全部服務，皆需在畫面上二次確認

第六章　實測數據

以下數字皆為 2026/10/20 在實機上量測所得，不是規格值或估算值。

6.1　兩顆模型的對照

我們在 .27 上安裝兩顆模型，以同一份 68,622 token 的長提示、在無快取的狀態下量測。

表 6-1　兩顆模型的實測對照

項目	通用模型 UD-IQ4_XS	程式專用模型 Coder IQ1_M
權重大小	87 GiB	54 GiB
匯入提示速度	1,011.6 tok/s	1,017.5 tok/s
生成回答速度	46.0 tok/s	45.4 tok/s
該次請求從硬碟讀取	70,563 MB	9.1 MB
載入後剩餘顯存	33,094 MiB	66,364 MiB
中文能力	正常	官方說明中文會出錯或重複迴圈

程式專用模型的速度與通用模型幾乎相同，但顯存用量少了一半。它的價值不在於更快，而在於騰出空間，讓同一台機器可以同時再跑另一個模型。由於它的中文能力明顯較弱，不適合用於中文資料清洗。

6.2　兩項與直覺相反的發現

第一，長提示的速度瓶頸在運算而不在硬碟。我們原本觀察到一次請求會從硬碟讀取 70 GB 資料，判斷硬碟是瓶頸；但程式專用模型把讀取量降到 9.1 MB 之後，匯入速度幾乎沒有變化。硬碟讀取確實發生，只是被其他運算重疊掉了，並未拖慢整體速度。

第二，提示快取的效益遠大於更換模型。同一份 68,622 token 的提示，在快取命中時只需 61 毫秒就讀完（其中 68,615 個 token 直接重用），整筆請求由 68.2 秒縮短為 24.7 秒。寫程式的用戶端每一輪送出的都是「同一份脈絡再加長一點」，因此快取命中率比模型本身的選擇更值得關注。

6.3　上下文長度與顯存

把上下文長度由 32,768 提高到 131,072，載入後的剩餘顯存由 33,026 至 33,416 MiB（同一設定連續量兩次）變為 33,094 MiB，幾乎沒有變化。這驗證了目前模型的上下文快取極小，可以放心給寫程式的用戶端使用長上下文。

6.4　畫面長度

依云碩網頁設計規範，內頁長度不應超過螢幕高度的 2.5 倍，手機不應超過 3.0 倍，以避免使用者需要一直往下捲動。

表 6-2　頁面長度實測（2026/10/21）

螢幕寬度	機器頁面長度	規範上限
1440（桌上型）	1.80 倍	2.5 倍
1280（筆記型）	2.03 倍	2.5 倍
375（手機）	2.71 倍	3.0 倍

附錄一　操作入口

表 A-1　操作入口

項目	內容
主控台網址	https://192.168.11.27:9400
連線條件	須位於公司內網，於外部使用時須先連線 VPN
瀏覽器警告	目前使用暫用自簽憑證，第一次開啟會出現安全警告，確認後即可進入
初始密碼	交接時另行提供，請於首次登入後變更
原始碼位置	Desktop/08.開發環境/claude/xConsole/code

常用的維運指令如下。

表 A-2　常用維運指令

用途	指令
變更登入密碼	在 .27 執行 `cd ~/xconsole && python3 xconsole_server.py --set-password`
單機模式部署	`bash deploy_standalone.sh <機器別名>`
加入一台機器	`bash deploy_agent.sh <機器別名> --seed`，再執行 `bash deploy_console.sh 27 <機器別名清單>`
准入公式自我檢查	`python3 agent/test_admit.py`（共 13 項）

附錄二　已知限制

以下事項在本版本尚未處理，列出以免使用者誤以為已具備。

表 B-1　已知限制

項目	目前狀況	影響
正式憑證匯入	尚無網頁上傳介面，需將憑證檔放入主控台機器指定目錄後重啟	換憑證需要具備主機操作權限的人員處理
服務使用報告	尚未實作月報產出功能	無法一鍵產出給主管檢視的使用報告
DGX Spark 實機納管	准入公式已通過自我檢查，但尚未接上實機驗證	DGX Spark 上的實際行為尚待確認
單一 GPU 的顯存歸屬	單一 GPU 機器無法逐一區分各程序的顯存，改依各欄位的預估需求比例分攤	同機多個模型並存時，各欄位顯示的顯存為估計值
系統記憶體量測	以程序常駐記憶體量測，含共用頁面，數值會隨時間浮動	單次讀數可能與長期平均有落差
未做過的檢測	尚未進行滲透測試與弱點掃描	對外開放前應先完成

本文件由云碩科技股份有限公司（xCloudinfo）編製　2026 年 10 月